Learning ROS 2 Humble: Core Concepts, Custom Interfaces, and Two Hands-On Solutions

Rama
Computer Science and Engineering, Manipal Institute of Technology
Formula Manipal

Abstract—This report summarises what I learned while getting started with ROS 2 Humble on Ubuntu 22.04 for Task 3 of the Driverless Taskphase. It explains the ROS 2 computation graph and its building blocks (nodes, topics, services, actions and parameters), when to use a topic and when to use a service, how custom interfaces, colcon and workspaces fit together, and what QoS settings do. It then walks through two solutions: a publisher and subscriber that work out a wheel's longitudinal speed from a custom TaskOne message, and a three-node palindrome checker built on a custom RT2String message. The rqt_graph output for both solutions is included.

Index Terms—ROS 2, Humble, computation graph, topics, services, actions, QoS, colcon, custom messages, rqt_graph.

I. Introduction

Robots are rarely one big program. A driverless car has software reading sensors, software making decisions and software driving the wheels, and all of it has to talk to each other. ROS 2 (Robot Operating System 2) is a framework built for exactly this: each piece runs as its own small program and they exchange data in a standard way, without needing to know about each other's internals.

For this task, Ubuntu 22.04 was set up first and ROS 2 Humble was then installed from the official documentation using the deb packages method. After that, the Humble beginner tutorials were worked through: the CLI tools (configuring the environment, turtlesim, rqt, nodes, topics, services, parameters, actions, rqt_console and launch files) and the client library tutorials (colcon, workspaces, packages, a Python publisher and subscriber, and custom msg and srv files).

II. The ROS 2 Computation Graph

The computation graph is the network of ROS 2 programs that are running and the connections between them. Its pieces are nodes, topics, services, actions and parameters, described below. Nodes find each other automatically at runtime through the underlying DDS middleware, so nothing needs to be hardwired. The tool rqt_graph draws this network live, which makes it the quickest way to check that nodes are actually connected the way you intended.

A. Nodes

A node is one program that does one job, such as reading a sensor or controlling a motor. A robot system is a collection of many such nodes. Useful commands are ros2 run ¡package¿ ¡executable¿ to start one, ros2 node list to see what is running and ros2 node info ¡node¿ to see its publishers, subscribers, services and actions.

B. Topics

A topic is a named channel for continuous, one-way data. A node publishes messages to a topic and any number of nodes can subscribe to it. The publisher does not know or care who is listening. Topics suit streams such as sensor readings or velocity commands. Handy commands are ros2 topic list, ros2 topic echo ¡topic¿, ros2 topic info ¡topic¿ and ros2 topic pub.

C. Services

A service is a request-and-response call. A client sends one request, the server does the work and sends back one response, and then the exchange is over. Services suit occasional, on-demand jobs such as "spawn a turtle" or "reset the simulation". They are used through ros2 service list, ros2 service type and ros2 service call.

D. Actions

An action is meant for long-running goals. It is built from topics and services and has three parts: a goal the client sends, feedback the server streams back while it works, and a final result. A goal can also be cancelled part-way. Rotating the turtle in turtlesim to a given angle is a typical example, because the turtle keeps reporting how far it still has to turn. ros2 action list and ros2 action send_goal are the main commands.

E. Parameters

A parameter is a setting that belongs to a node, such as a colour value or a speed limit, and that can be read or changed while the node runs. ros2 param list, get, set and dump are used for this. In turtlesim, for example, changing the background colour parameters changes the window colour straight away.

F. Topic vs. Service

Both move data between nodes, but they fit different situations (Table 1). A good rule of thumb is that if data keeps flowing and many nodes may want it, use a topic. If one node needs to ask another to do something and wait for a definite answer, use a service. If the job takes a while and the caller wants progress updates or the ability to cancel, use an action.

Table 1: Topic compared with service

	Topic	Service
Pattern	Publish / subscribe	Request / response
Direction	One way	Two way, one reply per request
Receivers	Many subscribers	One server
Timing	Continuous stream	On demand
Example	Wheel speed readings	"Reset the simulation"

III. Custom Interfaces

ROS 2 comes with common message types, but you can define your own in a .msg file (for topics) or a .srv file (for services). A message file is just a list of typed fields, for example two float32 values. When the package is built, ROS 2 generates Python and C++ code for it so nodes can import and use it like any built-in type.

In Humble, interfaces are generated by an ament_cmake package. That package holds the msg/ folder, lists the files in rosidl_generate_interfaces in its CMakeLists.txt, and declares the needed dependencies in package.xml. This is why each solution below has a separate message package alongside the node package: the nodes depend on the message package and import the generated type from it.

IV. colcon and Workspace Structure

A workspace is simply a folder where you build your own ROS 2 packages, and colcon is the tool that builds them. A workspace such as ros2_ws has four folders:

- src holds the source code of the packages. This is the only folder written by hand.
- build holds intermediate build files.
- install holds the finished packages, including setup.bash.
- log holds build logs.

The usual routine is to run colcon build from the workspace root and then source install/setup.bash so the terminal can see the new packages. A Python package contains a package.xml, a setup.py and a folder of the same name holding the node scripts, with each node registered under entry_points so that ros2 run can find it. The base ROS 2 installation is sourced first (source /opt/ros/humble/setup.bash) and the workspace is layered on top of it.

V. QoS Settings

Quality of Service (QoS) settings control how messages are delivered between a publisher and a subscriber. The main ones are:

- Reliability: reliable retries until a message arrives, while best effort sends once and accepts that some may be lost. Best effort suits fast sensor data where the latest value matters most.
- Durability: volatile gives late joiners nothing from the past, while transient local keeps the last messages so that a late subscriber still receives them.
- History and depth: keep last stores only the newest N messages (N is the depth), while keep all stores everything.
- Deadline, lifespan and liveliness: limits on how often data must arrive, how long a message stays valid, and how to tell whether a publisher is still alive.

The default profile is reliable, volatile, keep last with a depth of 10, which is what the simple publishers and subscribers here use. A subscriber only connects if the publisher offers at least the QoS the subscriber asks for. For example, a reliable subscriber will not receive from a best-effort publisher, so mismatched QoS is a common reason for "it should work but nothing arrives".

VI. Solution 1: Wheel Speed (TaskOne)

A. What was built

The task was to create a custom message TaskOne with two float values, angVel and radius, and a publisher and subscriber around it. The publisher node, wheel_publisher, asks the user in the terminal for the angular velocity and radius of the wheel and publishes them on the topic /taskone. The subscriber node, wheel_subscriber, listens on that topic and, whenever data arrives, works out the longitudinal speed of the wheel and prints it:

$$v = \omega\, r \tag{1}$$

where ω is the angular velocity (angVel) and r is the wheel radius. The nodes are in a Python package called taskone_py (started with ros2 run taskone_py publisher), and the TaskOne message sits in its own interface package.

B. What the graph shows

Fig. 1 is the rqt_graph view while both nodes were running. It is about as simple as a graph gets: one arrow from /wheel_publisher to /wheel_subscriber, labelled /taskone. That arrow confirms the nodes found each other and that data flows one way only. The extra node /rqt_gui_py_node_3566 is not part of the solution; it is just rqt itself.

VII. Solution 2: Palindrome Checker

A. What was built

The second task uses three nodes and a custom message RT2String that holds a single string.

- string_publisher (Node 1) keeps publishing strings typed in by the user, such as "AABAA" and then "BACAB", on /rt2_string.
- palindrome_checker (Node 2) receives each string, checks whether it reads the same forwards and backwards, and sends 1 (true) or 0 (false) to Node 3 on /palindrome_flag.

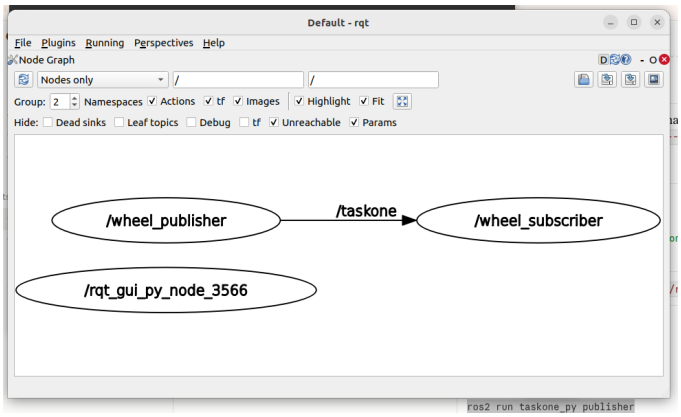


Figure 1: rqt_graph for Solution 1: /wheel_publisher sends TaskOne messages to /wheel_subscriber over /taskone.

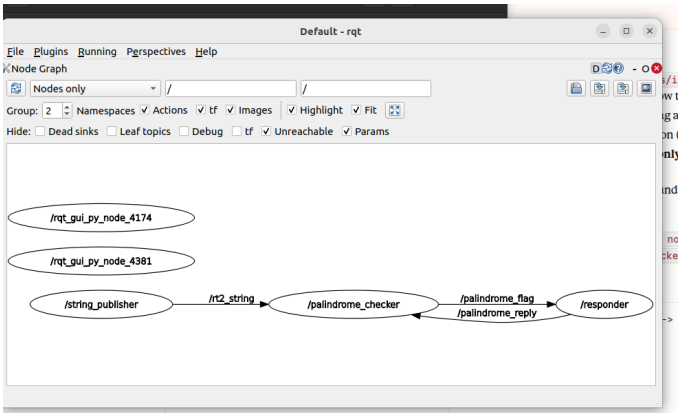


Figure 2: rqt_graph for Solution 2: strings flow from the publisher to the checker, and the checker and responder exchange the flag and the reply.

- responder (Node 3) receives that value and answers "Yes" for 1 or "No" for 0, sending the answer back to Node 2 on /palindrome_reply. The result is printed in the terminal.

B. What the graph shows

Fig. 2 shows the three nodes in a line. The arrow from /string_publisher into /palindrome_checker carries the input strings. From the checker, /palindrome_flag goes out to /responder, and /palindrome_reply comes back the other way. That pair of arrows is the interesting part: a two-way conversation built from two separate one-way topics, since a topic only ever flows in one direction. The two rqt_gui_py_node entries at the top are rqt itself; there are two because rqt was opened twice.

VIII. What I Learned

Seeing the graphs made a few things click. A topic is just a name, and the graph is the quickest way to check that publisher and subscriber agree on it, because a typo means two nodes silently never connect. Custom messages make a system feel like your own: TaskOne gives the wheel data proper named fields instead of squeezing it into a string. And since topics are one-way, a back-and-forth like the checker and responder needs a topic in each direction (a service would also have worked there, which is exactly the topic-versus-service choice discussed earlier).

IX. Conclusion

Both solutions ran as intended and the rqt_graph output matched the design each time. Together with the concepts covered above, they cover the core ideas needed to build larger ROS 2 systems, such as a driverless vehicle's pipeline from perception to control.

References

[1] Open Robotics, "ROS 2 Documentation: Humble," https://docs.ros.org/en/humble/

[2] Open Robotics, "ROS 2 Tutorials: Beginner CLI Tools and Client Libraries," ROS 2 Humble Documentation. https://docs.ros.org/en/humble/Tutorials.html